

Automated-review-rating-system

This project aims to automatically predicts the numerical ratings of the reviews given by the user. It uses Natural Language Processing (NLP), & Deep learning techniques for it.

Project overview and objective

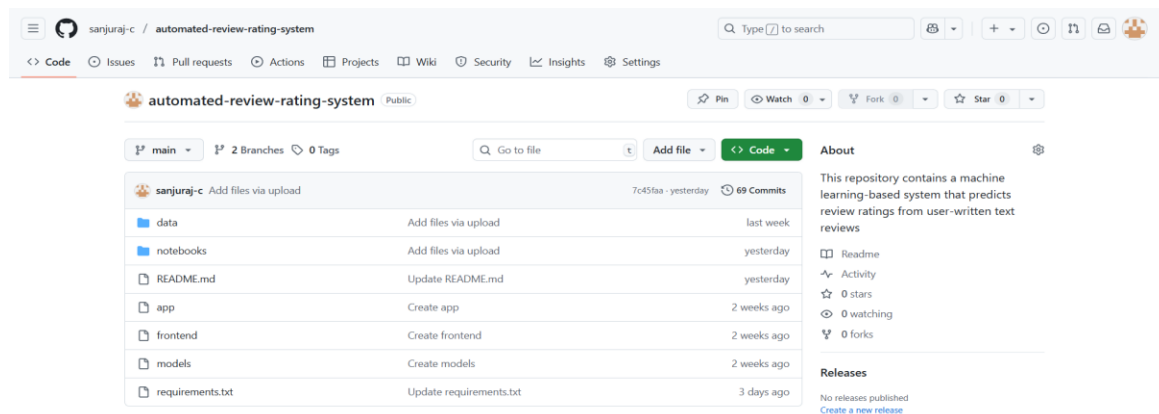
This project aims to build an automated review rating system that analyzes customer reviews and predicts the rating associated with each review. In many real-world scenarios, users provide detailed textual feedback, but numeric ratings may be missing, inconsistent, or biased. The goal is to train a Deep learning model that can understand the sentiment and content of a review and accurately classify it into one of the standard rating categories (e.g., 1 to 5 stars).

By processing and analyzing review text using natural language processing techniques, this system helps businesses quickly identify the sentiment of customer feedback, monitor product performance, and make data-driven decisions without manual intervention.

Environmental Setup

The project was implemented using Python for data processing and analysis due to its rich ecosystem of libraries. Development and testing were carried out in Jupyter Notebook, providing an interactive environment for writing and visualizing code outputs. The entire codebase, along with dataset references and updates, was version-controlled using GitHub for collaboration and backup.

GitHub Project Setup:



The screenshot shows the GitHub repository page for 'automated-review-rating-system' by user 'sanjuraj-c'. The repository is public and has 69 commits. The file list includes 'data', 'notebooks', 'README.md', 'app', 'frontend', 'models', and 'requirements.txt'. The 'About' section describes the repository as a machine learning-based system that predicts review ratings from user-written text reviews. The 'Releases' section indicates that no releases have been published.

File	Commit Message	Commit Time
data	Add files via upload	last week
notebooks	Add files via upload	yesterday
README.md	Update README.md	yesterday
app	Create app	2 weeks ago
frontend	Create frontend	2 weeks ago
models	Create models	2 weeks ago
requirements.txt	Update requirements.txt	3 days ago

Dataset description

To build the automated review rating system, data was gathered in two phases from various sources like Kaggle, Google search datasets, Data web scraping using API Key method in which, containing customer reviews along with corresponding ratings. The objective was to retain only the review text and rating fields after cleaning and preprocessing.

1. Amazon Fashion Reviews (<https://www.kaggle.com/datasets/fawadhossaini1415/amazon-fashion-800k-user-reviews-dataset>)

-This dataset comprises a large-scale collection of Amazon Reviews, gathered in 2023 by the McAuley Lab. Specifically, it focuses on the Amazon Fashion category, containing a total of 800K+ user reviews. It serves as a valuable resource for conducting sentiment analysis.

-Key Fields: Title, Review Text, images, asin, parent_asin, User_id, timestamp, Rating, helpful_vote, target, verified_purchase.

Preprocessing Steps

After collecting and cleaning the datasets, the following initial preprocessing steps were applied to prepare the review text and rating data for model training:

1. Data Loading and Inspection

Each dataset was loaded using pandas and inspected using functions like `.head()`, `.tail()`, `.sample()`, `.shape`, `.columns`, `.info()`, and `.describe()` to understand structure and content.

#Load

```
df = pd.read_csv('data.csv')
```

#View

```
df.head()      # First 5 rows
```

```
df.tail()      # Last 5 rows
```

```
df.sample(3)   # Random 3 rows
```

#Inspect

```
df.shape       # Rows and columns
```

```
df.columns     # Column names
```

```
df.info()      # Data types and nulls
```

```
df.describe()  # Summary stats
```

2. Handling Missing Values

- Columns that had a large number of missing values or were irrelevant to the analysis were dropped.
- Rows with missing entries in the review text or rating columns were removed to maintain data quality.

```
df.isnull().sum()          # Count nulls
df.dropna(inplace=True)    # Drop rows with any nulls
df['x'].fillna(df['x'].mean(), inplace=True) # Fill with mean
```

3. Dropping Irrelevant Columns

- Only the review text and rating columns were retained from each dataset.
- Other metadata like dates, user details, product configuration, etc., were considered unnecessary and discarded.

```
df.drop('x', axis=1, inplace=True) # Drop column
```

4. Duplicate Removal

- All datasets were checked for duplicate rows using `.duplicated()`.
- Any duplicate records found (if any) were removed to avoid data redundancy.

```
df.duplicated().sum() #finding duplicates
df.drop_duplicates(inplace=True) #removing duplicates
```

Review Text Processing

6. Emoji and Special Character Removal

- Emojis, symbols, and special characters were removed using regular expressions to focus only on meaningful textual content.

```
# Remove emojis
text = re.sub(r'[^\w\s,]', '', text.encode('ascii', 'ignore').decode('ascii'))
```

```
# Remove special characters and digits (keep letters and basic punctuation if needed)
text = re.sub(r'[^\A-Za-z\s]', '', text)
```

7. Stop Word Removal

-Common stop words (e.g., “the”, “is”, “and”) were removed using NLTK to reduce noise and retain only informative words.

#defining stopword removal function

```
stop_words = set(stopwords.words('english'))
```

```
punctuations = string.punctuation
```

```
def remove_stopwords(text):
```

```
    # Lowercase
```

```
    text = text.lower()
```

```
    # Remove punctuation
```

```
    text = ''.join([char for char in text if char not in punctuations])
```

```
    # Tokenize and remove stopwords
```

```
    words = text.split()
```

```
    filtered = [word for word in words if word not in stop_words]
```

```
    return ' '.join(filtered)
```

-stopwords used:

-From NLTK

-stop word count= 198

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', 'aren't', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', 'couldn't', 'd', 'did', 'didn', 'didn't', 'do', 'does', 'doesn', 'doesn't', 'doing', 'don', 'don't', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', 'hadn't', 'has', 'hasn', 'hasn't', 'have', 'haven', 'haven't', 'having', 'he', 'he'd', 'he'll', 'he's', 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', 'i'd', 'i'll', 'i'm', 'i've', 'if', 'in', 'into', 'is', 'isn', 'isn't', 'it', 'it'd', 'it'll', 'it's', 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'mightn', 'mightn't', 'more', 'most', 'mustn', 'mustn't', 'my', 'myself', 'needn', 'needn't', 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', 'shan't', 'she', 'she'd', 'she'll', 'she's', 'should', 'should've', 'shouldn', 'shouldn't', 'so', 'some', 'such', 't', 'than', 'that', 'that'll', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'they'd', 'they'll', 'they're', 'they've', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', 'wasn't', 'we', 'we'd', 'we'll', 'we're', 'we've', 'were', 'weren', 'weren't', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'won't', 'wouldn', 'wouldn't', 'y', 'you', 'you'd', 'you'll', 'you're', 'you've', 'your', 'yours', 'yourself', 'yourselves']

8. Lemmatization

-Words were reduced to their root forms (e.g., “running” → “run”) using WordNetLemmatizer, improving consistency across similar word forms.

#defining lemmatization function

```
lemmatizer = WordNetLemmatizer()
```

```
def lemmatize_text(text):
```

```
    words = text.split()
```

```
    lemmatized = [lemmatizer.lemmatize(word) for word in words]
```

```
    return ' '.join(lemmatized)
```

-Why Lemmatization Over Stemming?

Lemmatization was chosen over stemming because it provides more accurate and meaningful word forms by reducing words to their dictionary base form (lemma), whereas stemming often results in incomplete or non-existent words. For example, "running" becomes "run" through lemmatization, but stemming might reduce it to "runn". This helps retain better context and grammatical correctness, which is especially important for tasks involving sentiment analysis and text understanding.

9. Filtering Out Short and Long Reviews

-Reviews with fewer than 3 words or more than 100 words were removed to retain only meaningful and focused feedback.

Keep only reviews that have word count between 3 and 100

```
filtered_df = df[df['Review'].apply(lambda x: 3 <= len(x.split()) <= 200)].reset_index(drop=True)
```

10. Conflicting Review Removal

-Reviews that did not align with their ratings (e.g., positive text with a negative rating or vice versa) were manually identified and removed.

#checking conflicting reviews

```
from textblob import TextBlob
```

Get sentiment polarity for each review (-1 = negative, 1 = positive)

```
df['Polarity'] = df['Review'].apply(lambda x: TextBlob(x).sentiment.polarity)
```

```

# Optional: classify sentiment direction
def classify_sentiment(p):
    if p > 0.2:
        return 'positive'
    elif p < -0.2:
        return 'negative'
    else:
        return 'neutral'
df['Sentiment'] = df['Polarity'].apply(classify_sentiment)

# Define conflict conditions
conflict_mask = (
    ((df['Rating'] >= 4) & (df['Sentiment'] == 'negative')) |
    ((df['Rating'] <= 2) & (df['Sentiment'] == 'positive'))
)
# Get conflicting reviews for optional inspection
conflicting_reviews = df[conflict_mask]

# Remove conflicting reviews
cleaned_df = df[~conflict_mask].reset_index(drop=True)

```

11. Encode Labels

-convert the rating labels (e.g., 1-star, 2-star, ..., 5-star) from their original categorical/string form into numerical integers so they can be processed by the model.

```

label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['Rating'])
num_labels = len(label_encoder.classes_)

```

-LabelEncoder(): Assigns each unique rating a unique integer (e.g., "1 star" → 0, "2 star" → 1, etc.).
 -num_labels: Stores the number of unique rating classes for use in the output layer of the LSTM model.

12. Tokenization & Padding

-convert the text reviews into numeric sequences that the LSTM can understand, while ensuring all sequences have the same length.

```

max_vocab_size = 10000
max_seq_len = 128

```

```
tokenizer = Tokenizer(num_words=max_vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(train_texts)

train_sequences = tokenizer.texts_to_sequences(train_texts)
test_sequences = tokenizer.texts_to_sequences(test_texts)

train_padded = pad_sequences(train_sequences, maxlen=max_seq_len, padding='post',
truncating='post')

test_padded = pad_sequences(test_sequences, maxlen=max_seq_len, padding='post',
truncating='post')
```

- Tokenization: Maps each word to a unique integer ID.
- OOV Token: <OOV> represents words not seen during training.
- Padding & Truncating: Ensures all reviews are exactly max_seq_len tokens long.

Padding: Adds zeros at the end of shorter sequences.

Truncating: Cuts off longer sequences to fit the maximum length.

13. Create TensorFlow Datasets

- wrap the padded sequences and their labels into a TensorFlow Dataset for efficient batching and shuffling during training.

```
batch_size = 32
```

```
train_dataset = tf.data.Dataset.from_tensor_slices((train_padded,
train_labels)).shuffle(1000).batch(batch_size)
```

```
test_dataset = tf.data.Dataset.from_tensor_slices((test_padded,
test_labels)).batch(batch_size)
```

- Shuffle: Randomizes training examples for better learning.
- batch(batch_size): Groups data into batches for efficient processing by the LSTM model.

Visualizations Used

To better understand the data and guide modeling decisions, several visualizations were created during the exploratory data analysis (EDA) phase:

1. Countplots (Per Rating Class)

- Displayed the number of reviews for each rating class using count plots.
- Useful for comparing rating frequency across different datasets.

2. Pie Charts of Overall Sentiment Distribution

- Ratings were grouped into broader sentiment categories like Positive, Neutral, and Negative.
- Pie charts helped visualize the proportion of each sentiment group in the dataset.

Before and After Cleaning/Balancing Comparisons

- Visualizations were also used to compare the dataset before and after cleaning and balancing. This helped in understanding how much the data was affected by preprocessing steps, especially in terms of class distribution. It clearly showed how the ratings became more evenly distributed after balancing, making the dataset suitable for training.

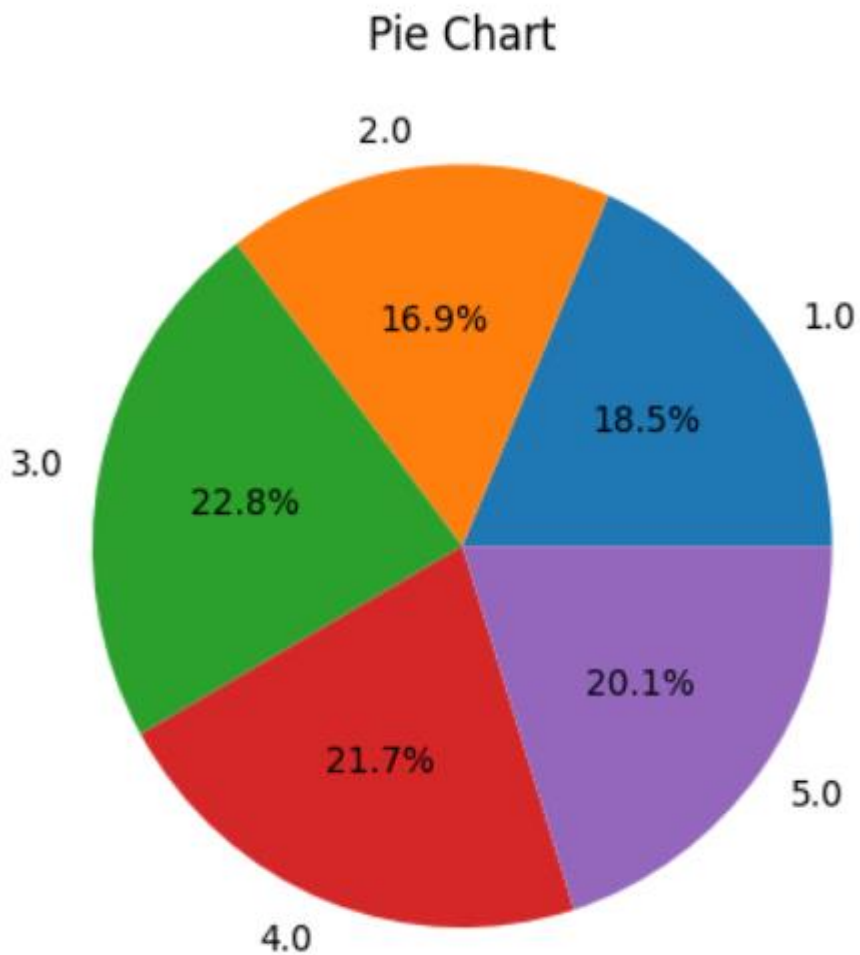
Final Dataset

-It has 683613 rows and 2 columns of 'Review Text' & 'Rating'.

Done visualization on the final data:

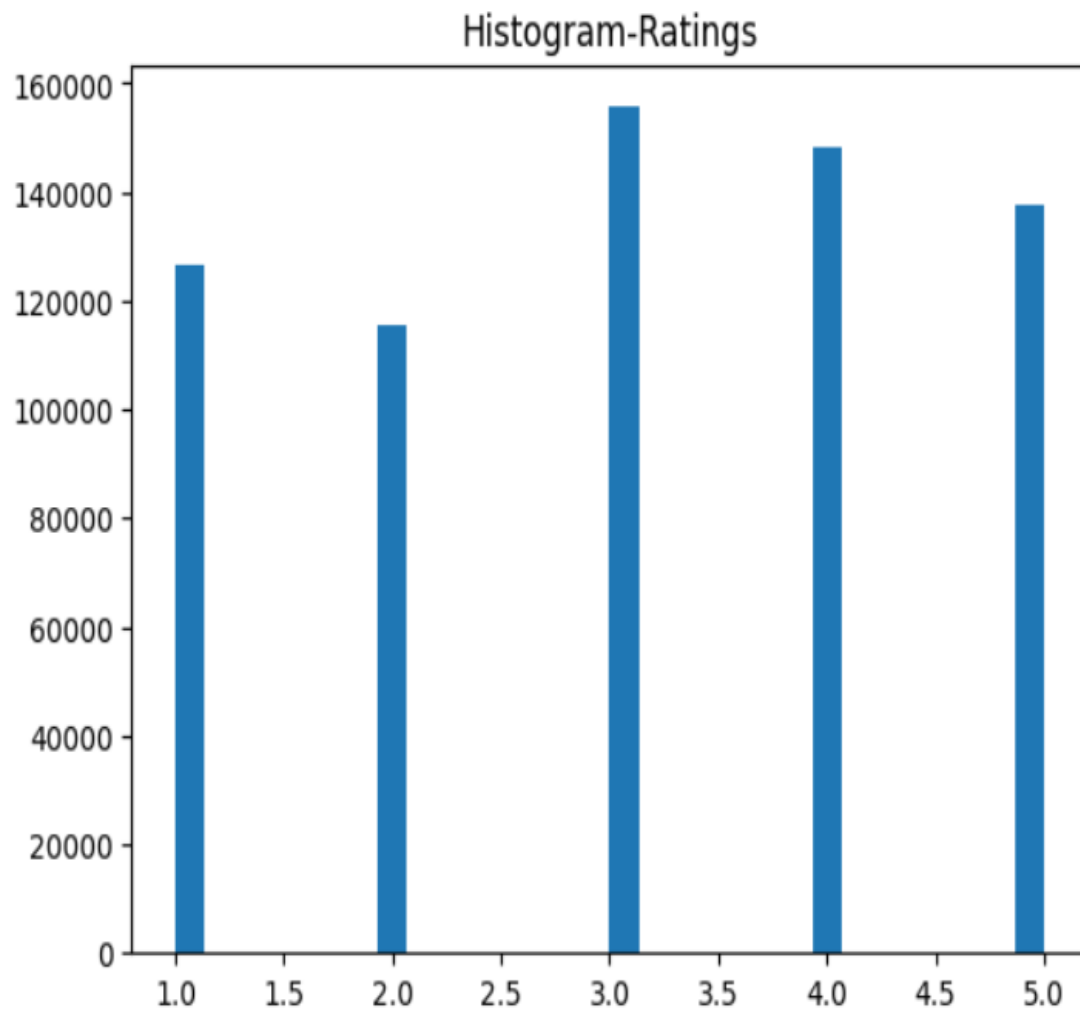
1. Pie Chart

```
rating_counts = df['Rating'].value_counts().sort_index()
plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```



1.Histogram

```
import matplotlib.pyplot as plt
rating_counts = unbalanced_df['Rating'].value_counts().sort_index()
plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```



Sample Reviews Per Rating :

```
for rating in sorted(balanced_df_final['Rating'].unique()):
    print(f"\n Rating: {int(rating)}")
    samples = balanced_df_final[balanced_df_final['Rating'] == rating].sample(3,
random_state=42)
    for i, review in enumerate(samples['Review'].values, 1):
        print(f"{i}. {review}\n")
```

output:

Rating: 1

- plastic shield big wont stay puy
- earring tiny like sized baby wish taken look gave gift thought would small like half inch diameter probably diameter daughter gracious insisted perfect know gracious get baby dont get teen adult
- really small maybe size x

Rating: 2

- material thicker expected snap bottom bothered year old returned chose one piece leotard built skirt
- much bigger advertised one cake ball missing
- first foremost say im pretty positive piece received white gold plated advertised shiny serious doubt white gold secondly chain provided stated previous reviewer pretty thin flimsy thirdly angel wing arrived ok condition dragon wing came kinda beat hole dent place theyre cute worth price

Rating: 3

- love color style however sizing run somewhat smaller advertised order would go size
- okay quite look wanted looking cute tee wear jeansbut didnt like look
- look super cool sadly back earring came one earring lost

Rating: 4

- nicely made smaller expected also less toe room wanted
- excellent gogglesbr great sealbr semifog proofbr little darkbr little bulky
- nice fit bit larger expected

Rating: 5

- love super cute fit well
- la mdida lindo color
- pair lace far good son struggle tying shoe difficult find size velcro shoe needed kind lace could stay tied also easy get onoff tried brand would come hole werent tight enough etc lace far winner stay place still easy get look great

Train-Test-split data for Training

To evaluate the model's performance effectively, the dataset was split into two parts: 80% for training and 20% for testing. The training set was used to fit the model, while the test set helped in assessing its performance on unseen data. This split ensures better generalization and avoids overfitting.

```
from sklearn.model_selection import train_test_split

train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['Review'].tolist(),
    df['label'].tolist(),
    test_size=0.2,
    stratify=df['label'],
    random_state=42
)
```

-Why Stratified Splitting?

Stratified splitting was used to ensure that the distribution of classes (ratings) remains consistent in both the training and testing sets. This is especially important when dealing with imbalanced datasets, as it helps maintain the proportion of each class across splits, leading to more reliable evaluation and better generalization of the model.

DL Model used- DistilBERT(Distilled Bidirectional Encoder Representations from Transformers)

<https://github.com/sanjuraj-c/automated-review-rating-system/blob/main/Deep%20Learning%20Model%20Used-Distil%20BERT.pdf>

Model Choice and Reason

For this project, we chose DistilBERT (Distilled Bidirectional Encoder Representations from Transformers), a lighter and faster variant of BERT, to perform review rating classification.

- Why DistilBERT?
 - BERT is one of the most powerful Transformer-based models for Natural Language Processing (NLP).
 - DistilBERT retains 97% of BERT's language understanding performance while being 40% smaller and 60% faster, making it suitable for production use.
 - Unlike traditional models such as LSTMs, DistilBERT uses the Transformer architecture with self-attention, which captures long-range dependencies in text more effectively.
 - It is pre-trained on large corpora, reducing the need for huge training data from scratch.
 - Perfect choice when we want real-time predictions without sacrificing too much accuracy.

Model Details:

- Base Model Used: distilbert-base-uncased (from Hugging Face Transformers library).
- Input Representation:
 - Tokenized reviews using WordPiece tokenizer.
 - Applied truncation & padding with a maximum sequence length of 64 tokens.

-Classification Head:

- A fully connected linear layer added on top of DistilBERT's pooled output.
- Number of output neurons = number of rating classes (based on LabelEncoder).
- Softmax activation ensures output probabilities across rating categories.

-Loss Function: CrossEntropyLoss (handled internally by DistilBertForSequenceClassification).

-Optimizer: AdamW (recommended for Transformer-based models).

-Device: Trained using GPU (cuda) when available.

Model Training Logic

The training process was carefully designed with mini-batches, evaluation, and early stopping for optimal performance.

Training Steps:

1. Dataset Preparation:

- Reviews were split into train (80%) and test (20%) sets using stratified sampling (to preserve class balance).
- Tokenized and converted into tensors using the Hugging Face tokenizer.
- Custom ReviewDataset class used to load data with PyTorch DataLoader.

2. Forward & Backward Pass:

- In each training step:
 - Input IDs, attention masks, and labels were passed into DistilBERT.
 - Model returned loss and logits.
 - Backpropagation was performed on loss.
 - Parameters were updated using AdamW optimizer with a learning rate of $2e-5$.

3. Validation after each Epoch:

- After training each epoch, the model was evaluated on the test set.
- Validation loss was computed to monitor generalization.
- Predictions were compared with true labels to calculate accuracy & metrics.

4. Early Stopping:

- A patience of 5 epochs was used.
- If validation loss did not improve for 5 consecutive epochs, training stopped to prevent overfitting.
- The best-performing model (lowest validation loss) was saved as best_model.pt.

5. Final Output:

- Saved the trained model weights, tokenizer, and label encoder for deployment.

Evaluation results and interpretation

1. Accuracy: 57.13%

2. Precision/Recall/F1-score:

Classification Report:

	precision	recall	f1-score	support
1.0	0.64	0.67	0.65	25337
2.0	0.44	0.36	0.40	23128
3.0	0.49	0.54	0.51	31123
4.0	0.56	0.48	0.52	29640
5.0	0.68	0.80	0.74	27495
accuracy			0.57	136723
macro avg	0.56	0.57	0.56	136723
weighted avg	0.56	0.57	0.57	136723

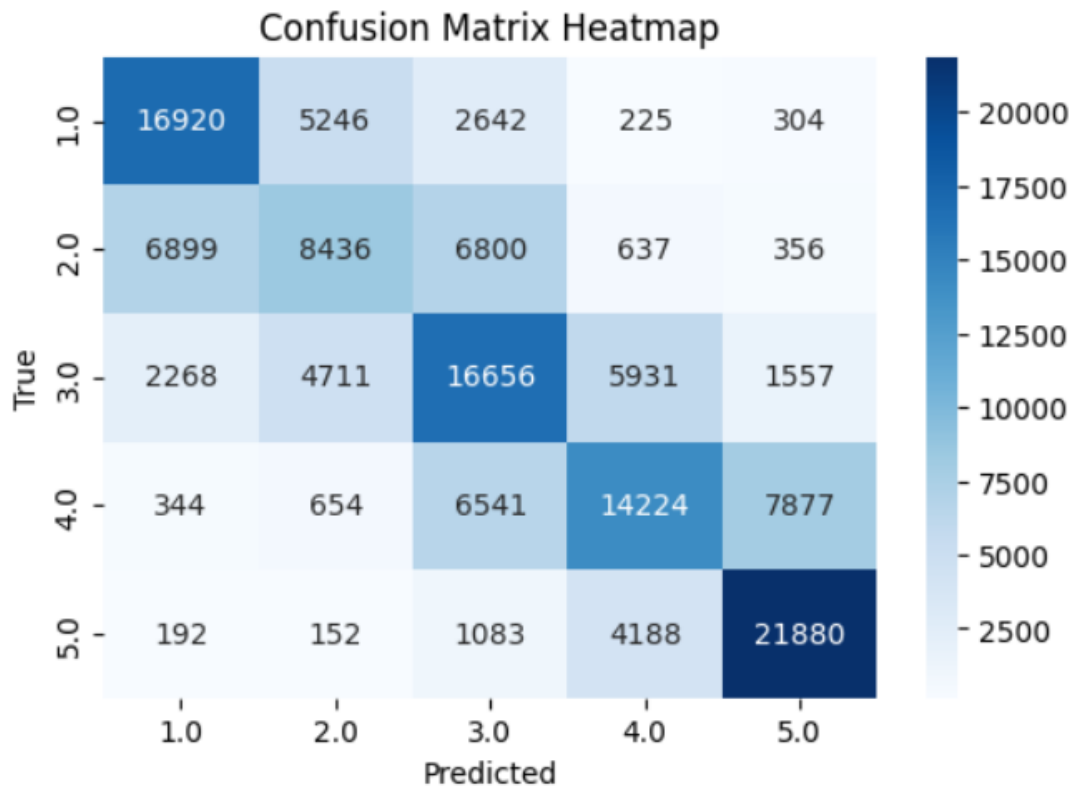
The performance of the model was evaluated using standard classification metrics:

- Accuracy: 0.57
→ The model correctly predicted ratings for about 57% of the test reviews.
- Precision: Measures how many of the predicted ratings were correct.
→ Precision values ranged from 0.44 (for rating 2) to 0.68 (for rating 5).
→ The model performed best for 5-star reviews, achieving the highest precision.
- Recall: Measures how many of the true ratings were correctly identified.
→ Recall was highest for 5-star reviews (0.80), meaning the model was very good at capturing positive reviews.
→ Recall was lowest for 2-star reviews (0.36), indicating the model struggled with identifying less frequent, ambiguous negative reviews.
- F1-Score: The harmonic mean of precision and recall.
→ F1-scores ranged between 0.51 (for 3-star reviews) and 0.74 (for 5-star reviews).
→ Overall weighted average F1-score was 0.57, aligning with accuracy.

Classification Report Insights

- Strong Performance on 5-Star Reviews:
The model is very reliable at identifying positive feedback, with precision = 0.68, recall = 0.80. This suggests that positive sentiments in reviews are more easily captured by the model.
- Weak Performance on 2-Star Reviews:
The model struggled to classify 2-star reviews correctly (precision = 0.44, recall = 0.36). This may be due to the ambiguous language used in moderately negative reviews and the smaller number of such samples in the dataset.
- Moderate Results for 3 and 4-Star Reviews:
These mid-range reviews had average precision/recall (~0.50–0.56). The overlap in language between neutral and moderately positive/negative reviews could be a reason.

3.Confusion matrix



Confusion Matrix Interpretation

- **Diagonal Dominance:** The confusion matrix shows high values along the diagonal (e.g., 16,920 for 1-star, 21,880 for 5-star), meaning the model makes a fair number of correct predictions.
- **Imbalance Effect:** Higher accuracy for 1-star and 5-star reviews suggests the model is biased towards extreme sentiments, while struggling with middle-ground ratings.

